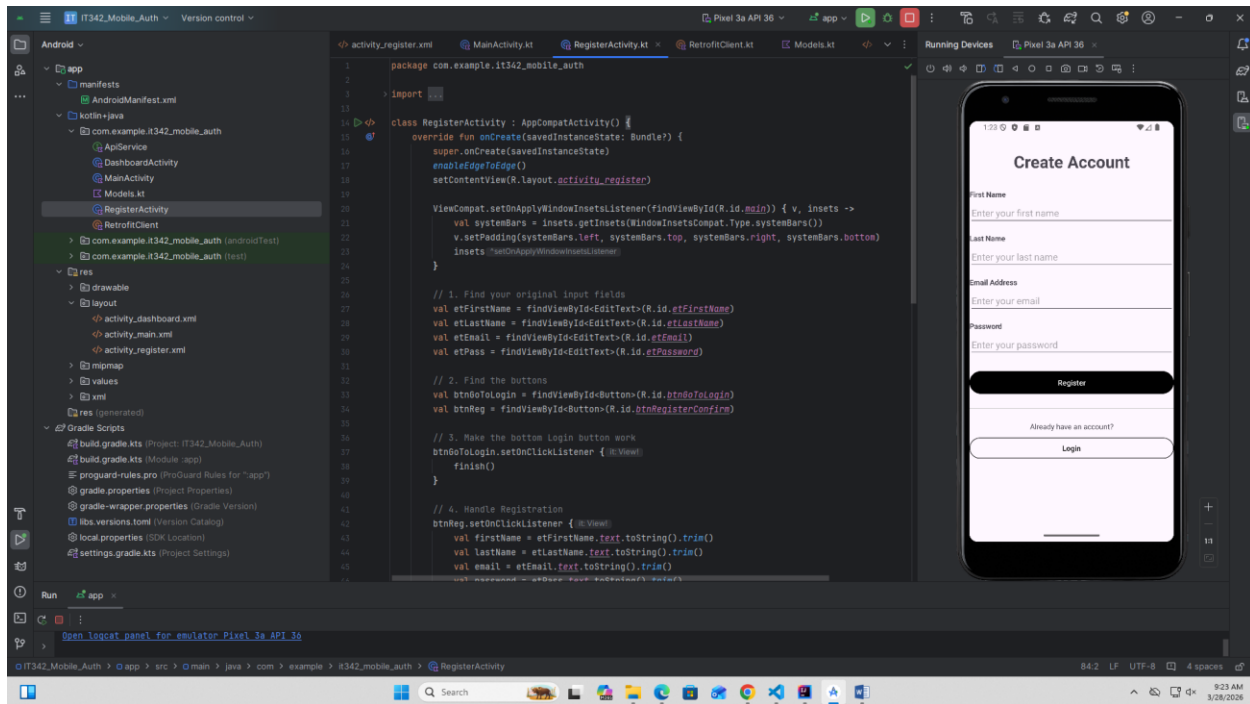
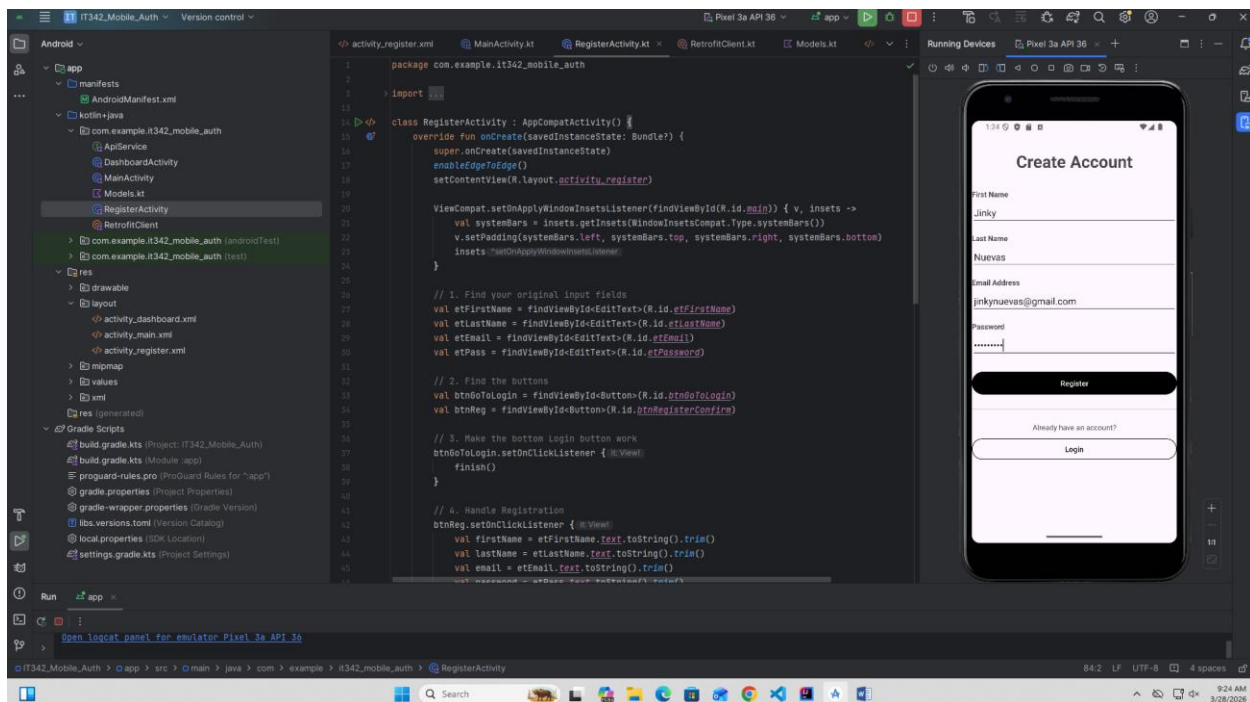
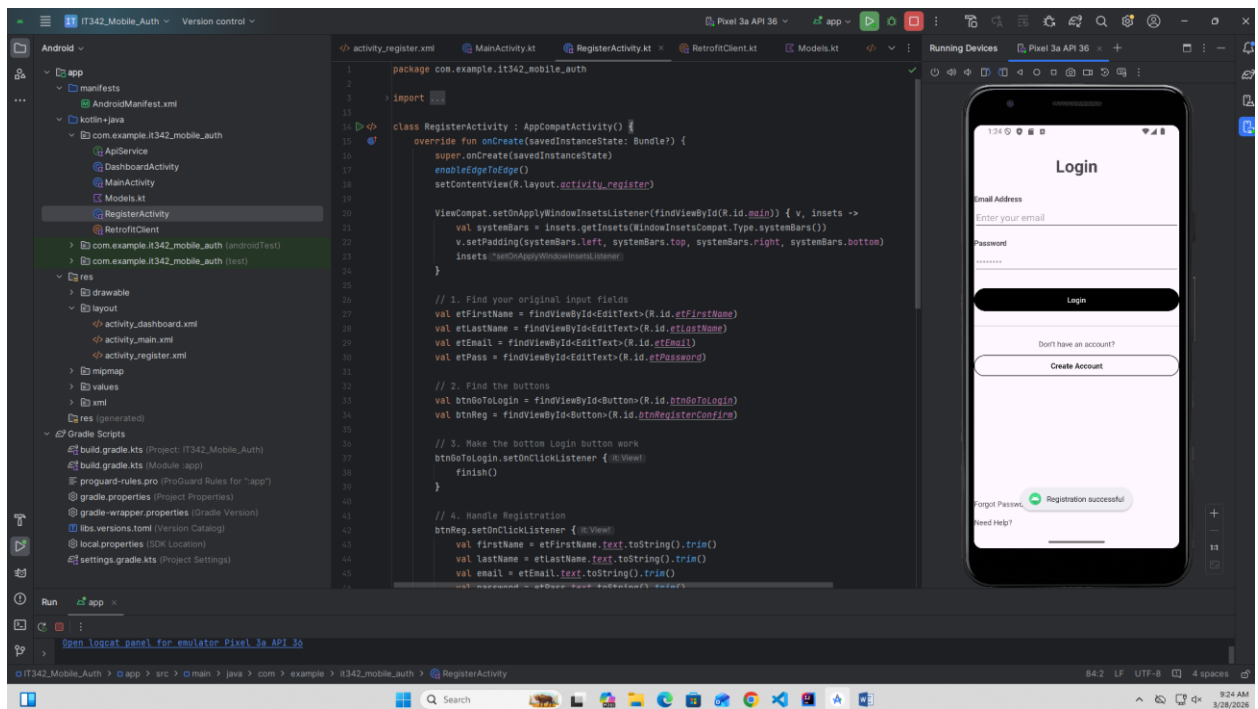


Registration Page

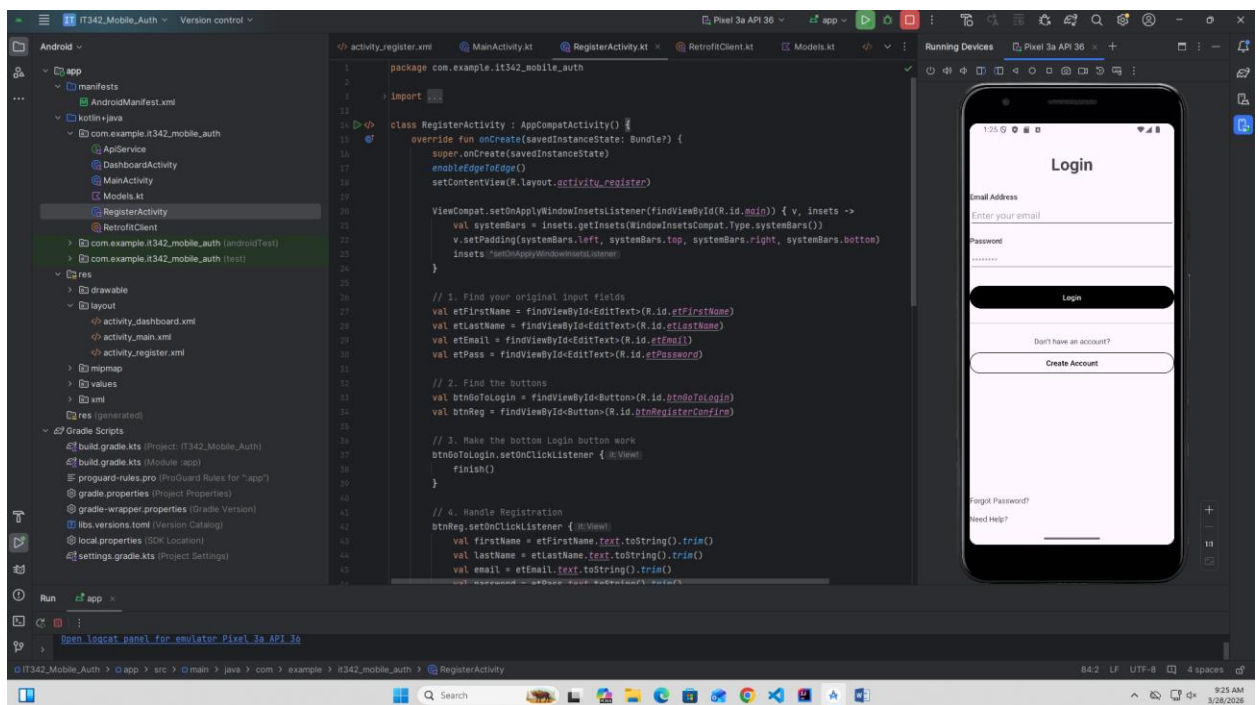


Successful User Registration

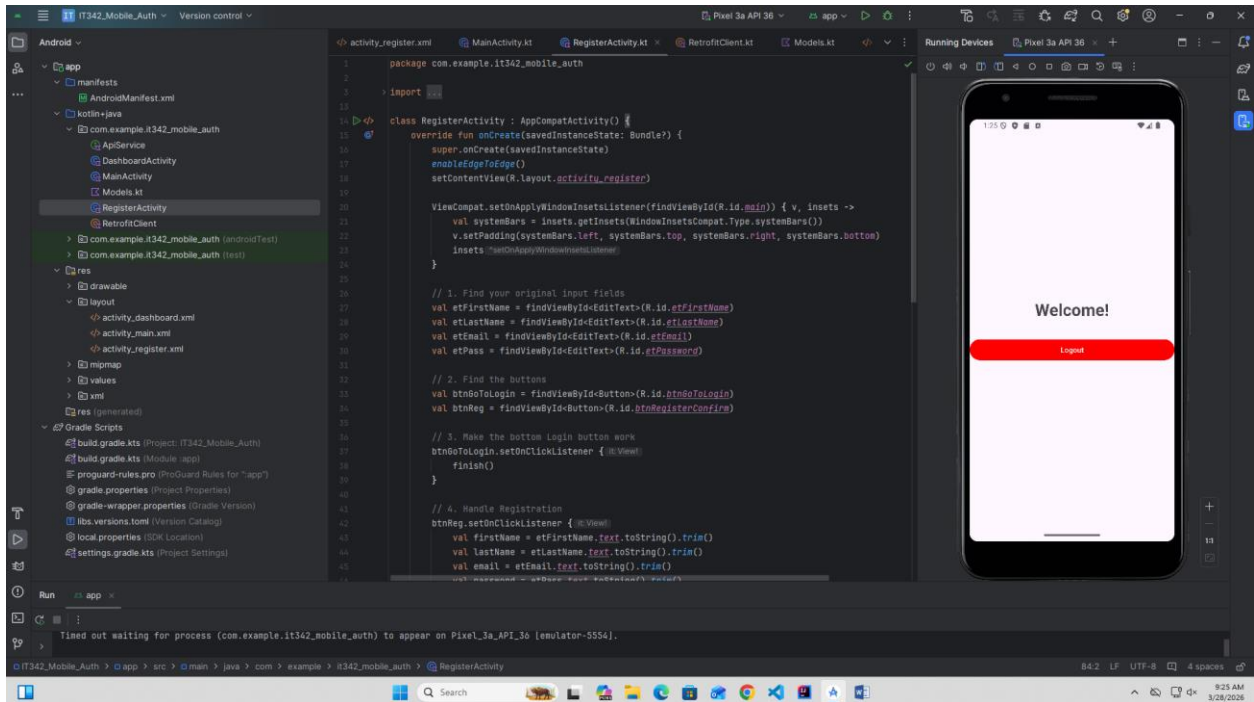




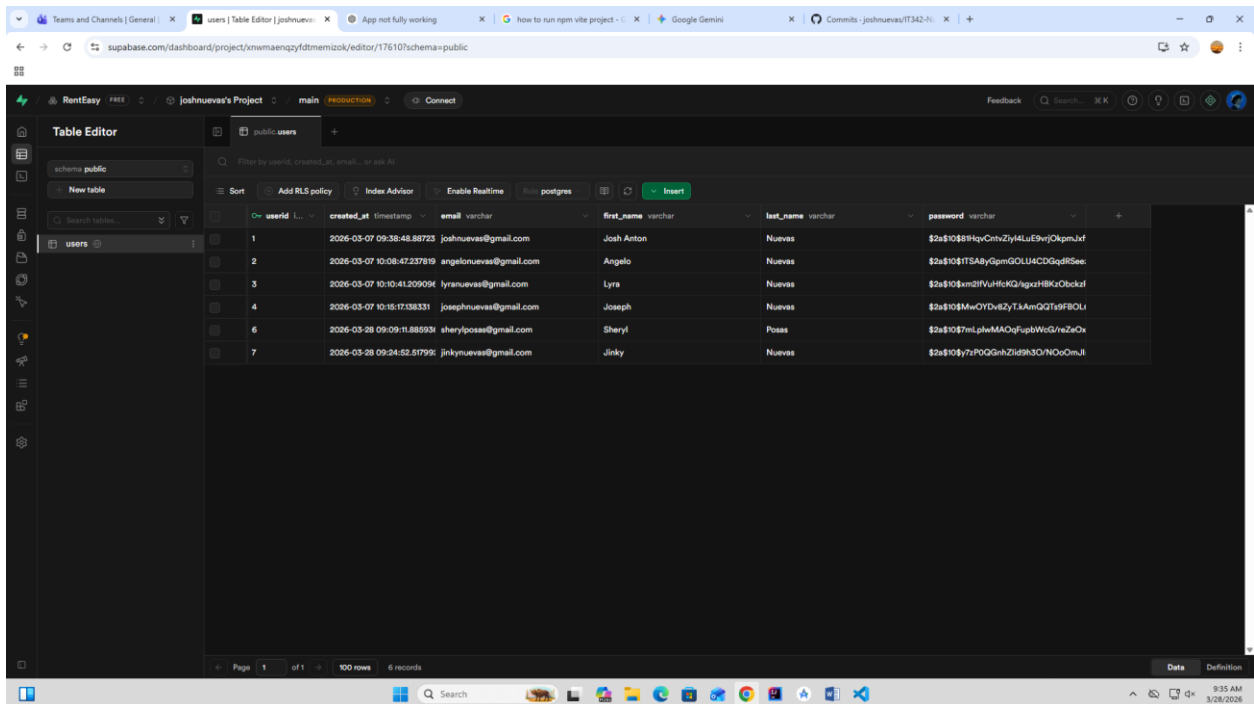
Login Page



Successful Login



Database Record Showing the Registered User



Final Commit: <https://github.com/joshnuevas/IT342-Nuevas-RentEasy/commit/eb3ebb5f0540c38f5cac90db46dc4ffa977eef61>

How Registration Works The registration process is designed to securely onboard new users for their rental journey while preventing duplicate accounts and protecting sensitive data.

- **Data Collection & Validation:** The system collects the user's first name, last name, email, and password. Basic input requirements are immediately enforced on the frontend using HTML5 validation.
- **Duplicate Prevention:** Before creating an account, the backend service queries the database (`findByEmail`). If the email is already registered, the process is aborted with an "Email already exists" error.
- **Secure Storage:** Passwords are never stored in plain text. The system utilizes **BCrypt hashing** to securely encrypt the password before persisting the new user record to the database.

How Login Works The login process verifies user credentials and establishes a secure, authorized session.

- **Credential Verification:** Users attempt to log in using their registered email and plain-text password.
- **Authentication:** The system retrieves the account via the provided email. It then uses `passwordEncoder.matches()` to securely compare the inputted raw password against the BCrypt hash stored in the database.
- **Authorization:** Upon successful authentication, the backend generates a **JSON Web Token (JWT)**. This token is sent back to the client application and used to securely authorize all future requests.

API & Database Integration The authentication flow is powered by a RESTful API backend and a cloud-hosted database.

- **REST Endpoints:** The backend exposes two primary endpoints to handle authentication traffic:
 - `POST /api/auth/register`: Receives new user data, hashes the password, and creates the account.
 - `POST /api/auth/login`: Verifies credentials and generates the session JWT.
- **Supabase / PostgreSQL:** User data is persisted in a PostgreSQL database hosted on **Supabase**. The system relies on a single `users` table containing the following columns: `userid` (Primary Key), `email` (Unique), `password`, `first_name`, `last_name`, and `created_at`.